



OBJECT ORIENTED ANALYSIS AND DESIGN

Ch - 7

A study material for the students of GLS University

What are Objects?

- A discrete entity with a well-defined boundary that encapsulates state and behavior
- An instance of a class
- Objects combine data and function in a cohesive unit
- Objects hide data behind a layer of functions called operations
- Hiding the data part of an object behind this layer of operation is known as encapsulation or data-hiding
- Every object is uniquely identifiable

Properties that are common to all objects :

- 1. Identity**
- 2. State**
- 3. Behavior**

1. Identity

- unique existence in time and space, what makes it different from all other objects
- Example, the printer's serial number
- A serial number is a great way to specify the identity of a physical object, and we use a similar principle, the idea of an object reference, to specify the identity of every one of the software objects we will be working with in OO analysis and design
- Of course, in the real world not every object has a serial number, but they still have unique identities

2. State

- Determined by the attribute values of an object at a particular point in time
- For example list of the states the printer can go through
- You can see from this how an object's state depends on the values of its attributes

Object state	Class attribute	Attribute value for object
On	power	on
Off	power	off
OutOfBlackInk	blackInkCartridge	empty
OutOfColourInk	colourInkCartridge	empty

3. Behavior

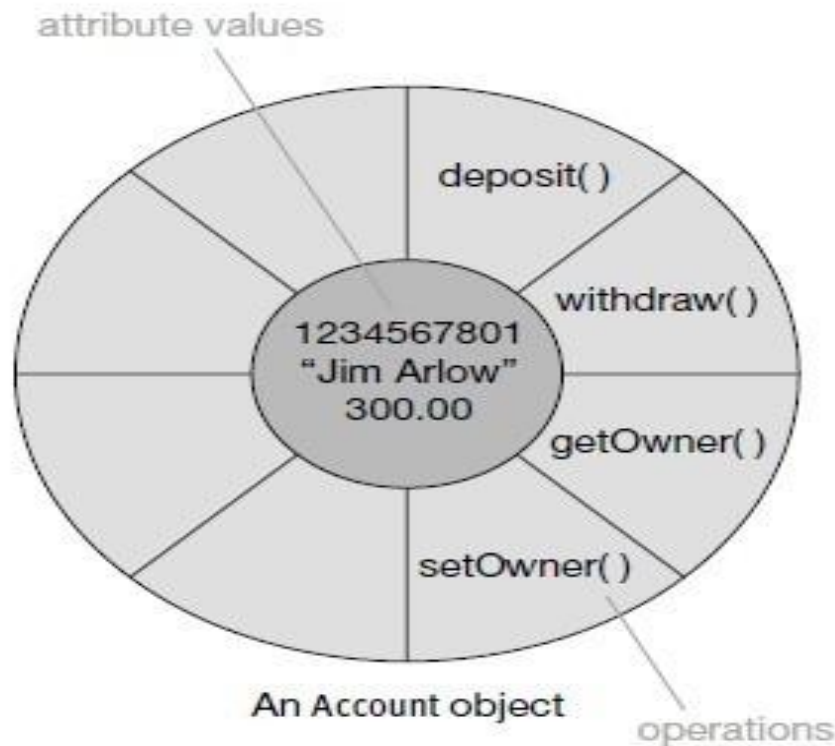
- Operations in analysis, and as methods in design
- Invoking an operation or method on an object will often cause a change in the values of one or more of its attributes, and this may constitute a **state transition**
- A method is an implementation of an operation
- This is a meaningful movement of the object from one state to another state
- For example – there are certain things the printer can do for me:
 - switchOn()
 - switchOff()
 - printDocument()
- An operation is the specification of a piece of behavior.
An implementation of that behavior is called a method

Encapsulation

Encapsulation, or data-hiding, is one of the primary benefits of OO programming

For example, if the balance is negative, then we may say that object is in the state **overdrawn**. As the balance changes from negative to zero, the object makes a significant change in its nature – it makes the state transition from the state overdrawn to the state **empty** and Account object's balance becomes positive, it makes another state transition from the state empty to the state **inCredit**

The behavior of any object is basically “what it can do” for us, and you can see that the object in Figure :



Messaging

- Objects generate system behavior by sending messages to each other over links. This is collaboration
- Objects collaborate to perform the functions of the system
What this means is that objects form links to other objects and send messages back and forth along those links
When an object receives a message, it looks at its set of operations to see if there is an operation whose signature matches the message signature

These signatures comprise the message (or operation) name, parameter types, and return value

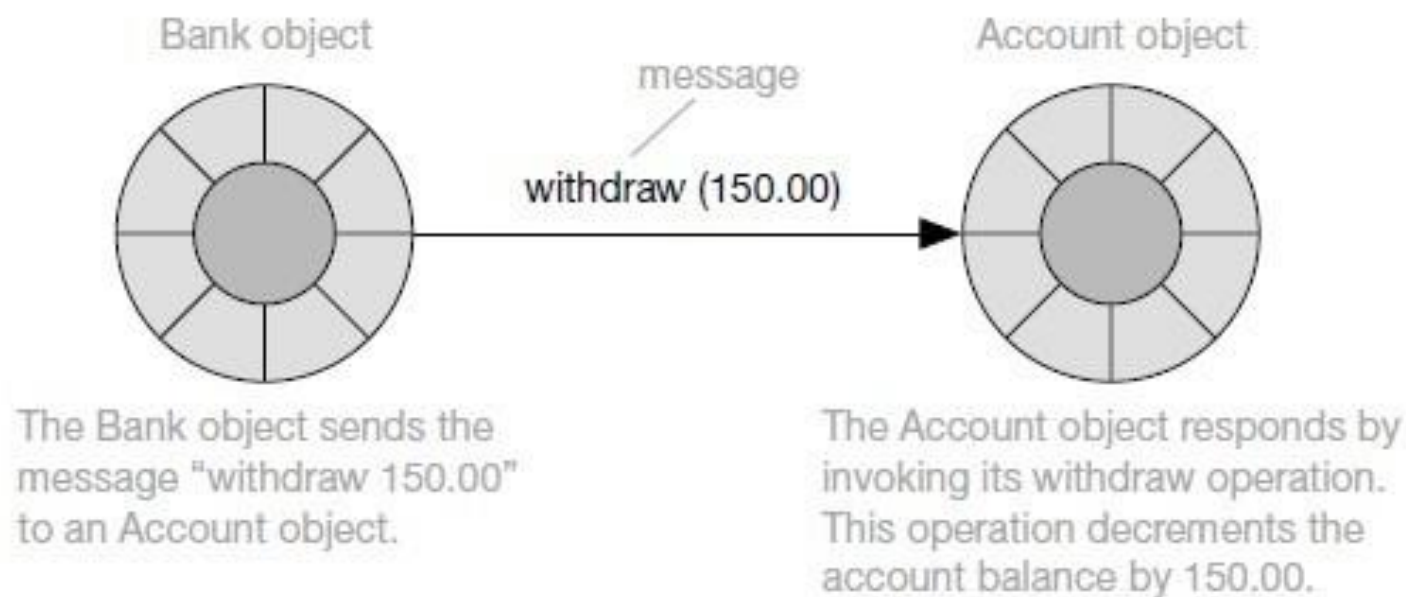
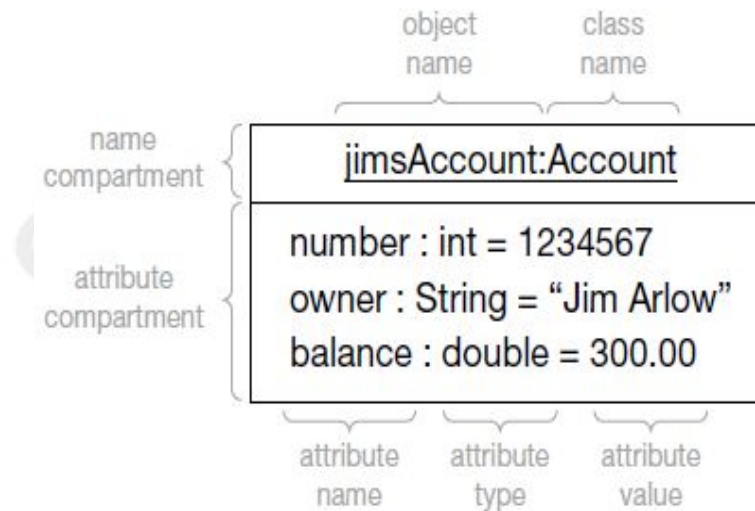


Figure 7.2

UML object notation

- The UML object icon is a box with two compartments:
- The top compartment contains the object identifier, which is *always underlined*
- The underline- defines - whether a modeling element is a class or an object
- Object



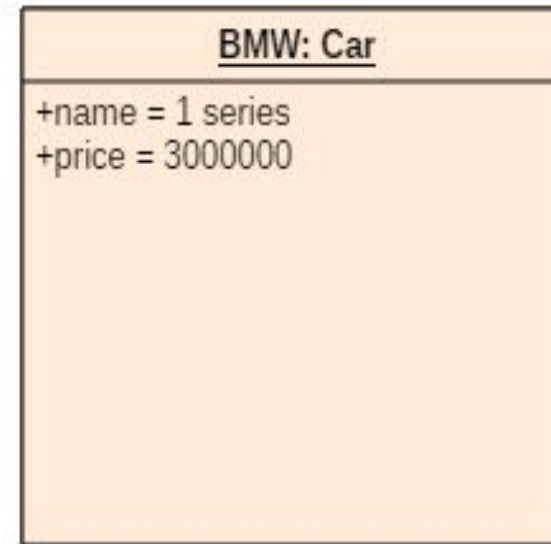
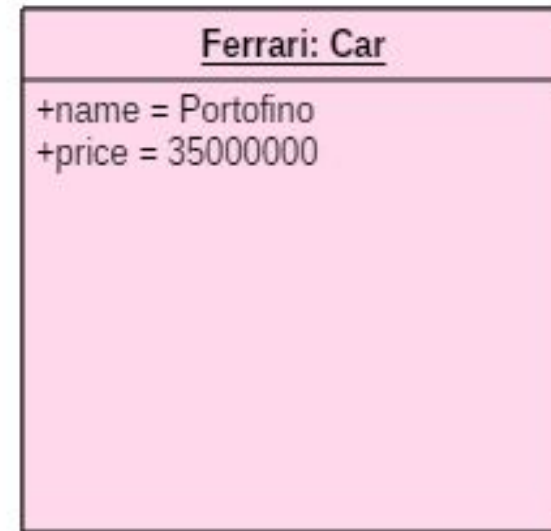
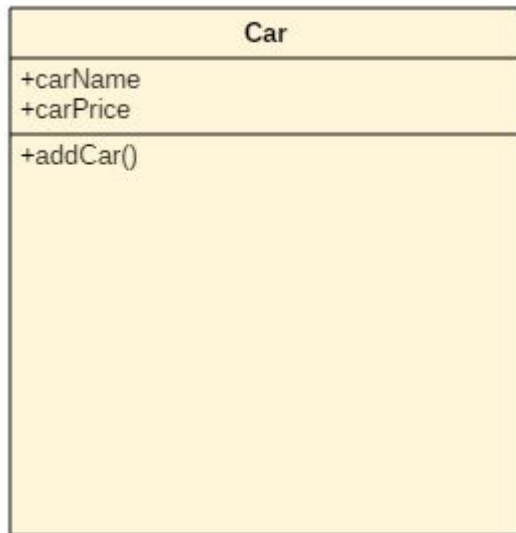
The object identifier can consist of any of the following:

1. **The class name alone** – e.g. **:Account**. This signifies that you have an anonymous object, or instance of that class
Anonymous objects are often used when there is only one object of a particular class on a given diagram
2. **The object name alone** – e.g. **jimsAccount**. This identifies a specific object but doesn't identify which class it belongs to.
This notation can be useful in very early analysis when you haven't yet discovered all the classes
2. **The object name concatenated with the class name**, separated from each other by a colon. You may read the colon as, "is an instance of class"

- **Objects** are usually named in mixed upper and lowercase, starting with a lowercase letter. Special characters, such as spaces and underscores, are avoided. This is known as **lowerCamelCase**
- **Attributes** may be optionally shown in the lower compartment of the object icon. Those attributes that you choose to show *must be named, and* may have an optional type and value. Attributes are also named in **CamelCase** starting with a lowercase letter.

Object attribute values

- → Each object attribute value has the following form:
name : type = value
- → You may choose to show all, some, or none of the object attribute values, depending on the purpose of the diagram
- → To keep object diagrams simple and clear, you may choose to omit attribute types as they are already defined in the class of the object

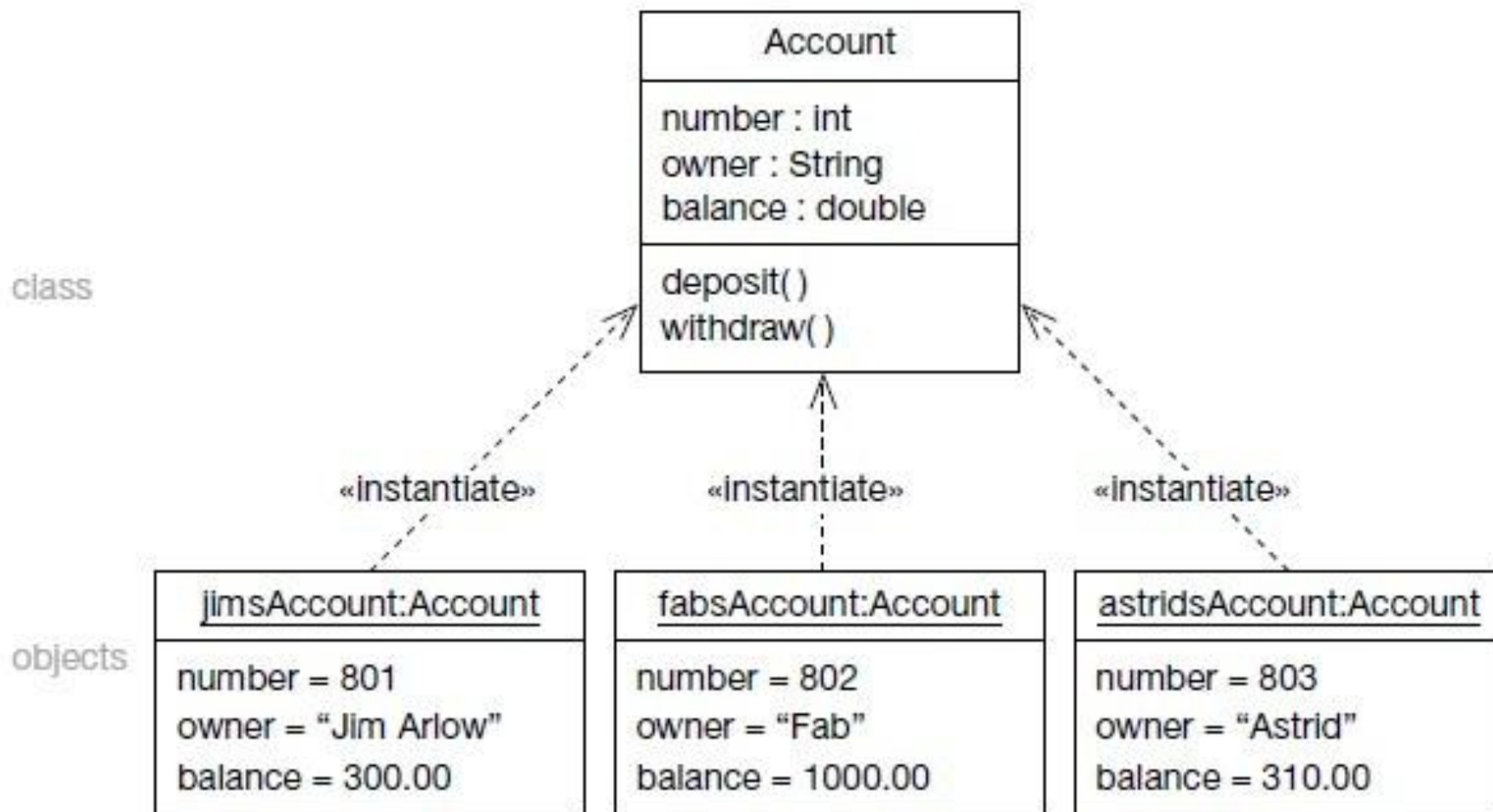


What are classes?

- A class describes the features of a set of objects
- A class defined as, “The descriptor for a set of objects that share the same attributes, operations, methods, relationships, and behavior”
- Every object is an instance of exactly one class

Classes and objects

- The relationship between a class and objects of that class is an **«instantiate» relationship**
- The dotted arrow is actually a dependency relationship that has been given a special meaning by the stereotype «instantiate»
- Anything inside guillemots («...») is known as a stereotype, and stereotypes are one of the three UML extensibility mechanisms
- A stereotype is a way of customizing modeling elements – a way of creating variants with new semantics. In this case, the stereotype «instantiate» turns an ordinary dependency into an instantiation relationship between a class and objects of that class



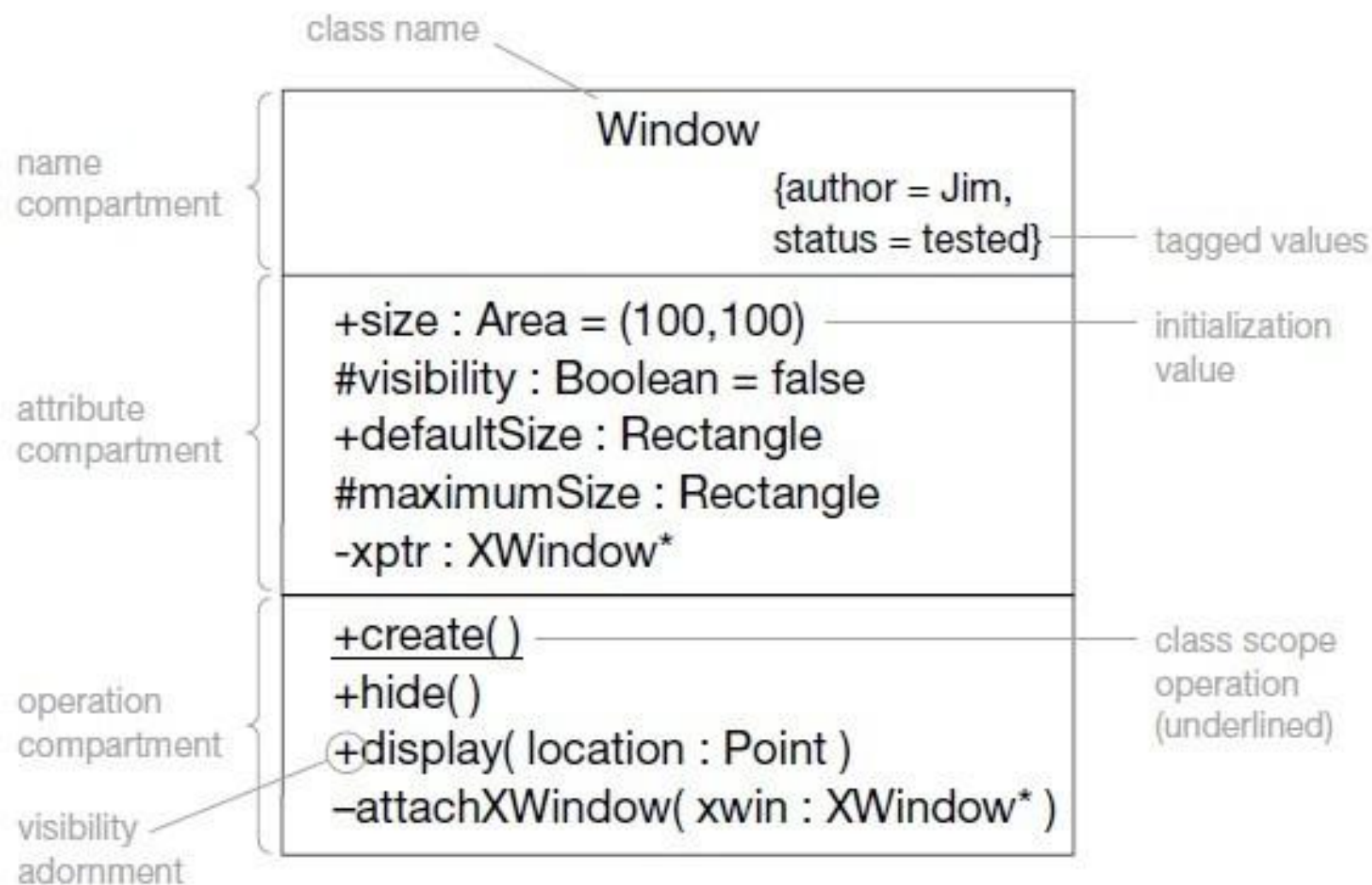
Class instantiation

- Instantiation is the creation of new instances of model elements. In this case we are instantiating objects from classes – we are creating *new instances* of classes
- In most OO programming languages, there are special operations, called constructors, which really belong to the class itself rather than the objects of that class
- The purpose of constructor operations is to create new instances of the class
- The constructor allocates memory for the new object, gives it a unique identity, and sets the initial attribute values for the object. It also sets any links to other objects

UML class notation

- The visual UML syntax for a class is very rich, and to make it manageable it is important to apply the UML notation of optional adornments
- The only mandatory part of the visual syntax is the name compartment with the class name in it
- All the other compartments and adornments are optional

Syntax: with example



Name compartment

- naming convention for classes

1. Class name is in UpperCamelCase –

- It begins with an uppercase letter, and then is in mixed upper and lowercase, with each word beginning in uppercase
- Special symbols such as punctuation marks, dashes, underscores, ampersands, hashes, and slashes are *always avoided*
- Including them in class names, or the names of any other modeling element for that matter, can lead to unexpected consequences when code or HTML/XML documentation is generated from the model

2. Avoid abbreviations *at all costs*

- *Class names should always reflect the names of real-world entities without abbreviation*
- *For example, FlightSegment is always preferable to FltSgmnt, DepositAccount is always preferable to DpstAcctnt*

3. If there are domain-specific acronyms (CRM- Customer Relationship Management) that are in common use and will be understood by all, then you may use them

Attribute compartment

- The only mandatory part of the UML attribute syntax is the **attribute name**

1. **Visibility**
2. **Type**
3. **Multiplicity**
4. **Initial Value**

Advance attribute syntax :



Visibility

Visibility controls access to the features of a class

Table 7.3

Adornment	Visibility Name	Semantics
+	Public visibility	Any element that can access the class can access any of its features with public visibility
–	Private visibility	Only operations within the class can access features with private visibility
#	Protected visibility	Only operations within the class, or within children of the class, can access features with protected visibility
~	Package visibility	Any element that is in the same package as the class, or in a nested subpackage, can access any of its features with package visibility

Type

An attribute's type may be another class or a primitive type

Primitive Type	Semantics
Integer	A whole number
UnlimitedNatural	A whole number ≥ 0 Infinity is shown as *
Boolean	Can take the value true or false
String	A sequence of characters String literals are quoted, e.g. "Jim"

Multiplicity

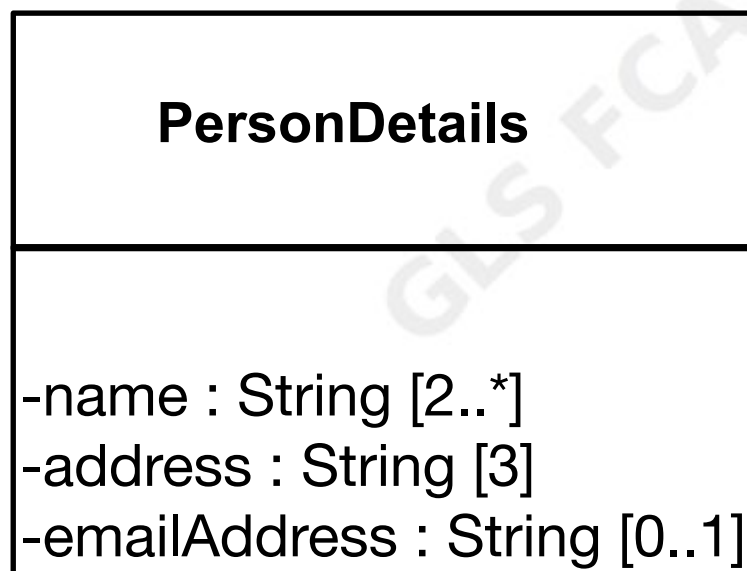
Multiplicity defines **how many instances** of a type can be associated with a single attribute. It's commonly used when an attribute represents **a collection (array, list, set)** rather than a single value.

How many objects of one class can be associated with an object of another class?

Multiplicity	Meaning	UML Notation
1	Exactly one	<code>name: String</code>
0..1	Zero or one	<code>nickname[0..1]: String</code>
0..*	Zero or more (optional list)	<code>emails[0..*]: String</code>
1..*	One or more (mandatory list)	<code>phones[1..*]: String</code>
n..m	Between n and m (inclusive)	<code>scores[2..5]: int</code>
*	Unlimited (same as 0..*)	<code>tags[*]: String</code>

Example :

UML Notation	Description
<code>phoneNumbers[0..*]: String</code>	Zero or more phone numbers (optional list)
<code>email[1]: String</code>	Exactly one email address
<code>children[1..*]: Person</code>	At least one child (must be a list of <code>Person</code>)
<code>grades[2..5]: int</code>	Between 2 and 5 grades
<code>aliases[*]: String</code>	Any number of aliases



name is composed of two or more Strings
 address is composed of three Strings
 emailAddress is composed of one String or null

Multiplicity

Collection

- If the multiplicity expression results in an integer greater than 1, then you are specifying a collection of
- the type For example, colors : Color[7] - collection of seven color objects

Null values

- When an attribute references null, this means that the object it points to has not yet been created or exist

The initial value allows you to set the value of an attribute at the point of object creation

Advance attribute syntax

You can also extend the specification of an attribute by postfixing the attribute with tagged values for example,

Syntax:

«stereotype» attribute { tag1 = value1, tag2 = value2, ... }

Example:

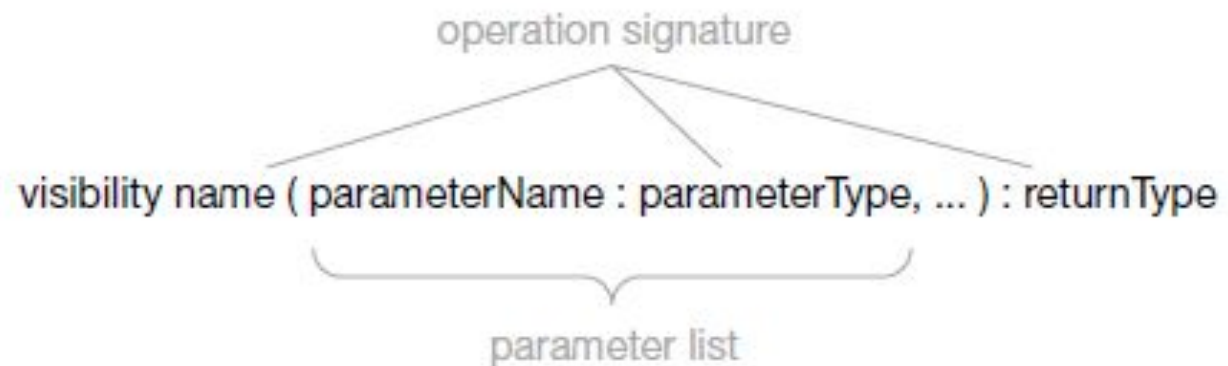
address {addedBy="Jim Arlow", date="20MAR2001"}

Operation Compartment

- An operation signature comprises its name, the type of all its parameters, and its return type
- Operations are functions that are bound to a particular class.

As such, they have all of the characteristics of functions:

- name
- parameter list
- return type
- Syntax :



Parameter direction

- Operation parameters can be given a direction
- Syntax:
Operation(in p1:Integer, inout p2:Integer, out p3:Integer, return p4:Integer, return p5:Integer)
- If the direction is not specified, then it defaults to in
- For example
maximumValue = max(a,b)

Normal UML operation :

maximumValue(a:Integer, b:Integer):Integer

With parameter direction

maximumValue(in a:Integer, in b:Integer):Integer

Parameter	Semantics
in p1:Integer	Default The operation uses p1 as an input parameter P1 is not changed by the operation
inout p2:Integer	The operation accepts p2 as an input/output parameter P2 may be changed by the operation
out p3:Integer	The operation uses p3 as an output parameter P3 may be changed by the operation
return p4:Integer	The operation uses p4 as a return parameter The operation returns p4 as one of its return values
return p5:Integer	The operation uses p5 as a return parameter The operation returns p5 as one of its return values

Parameter default values

- You can give an operation parameter a default value
- When the operation is called, if no value is given for that parameter, its default value will be used

Canvas
<pre>drawCircle(origin: Point = Point(0,0), radius:Integer) drawSquare(origin:Point = Point(0,0), size:Dimension)</pre>

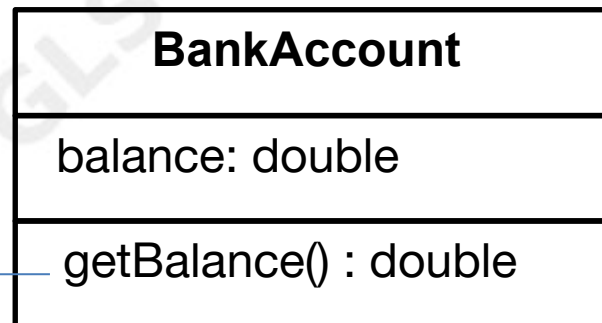
Advanced operation syntax

- You can extend the semantics of operations by prefixing them with stereotypes and postfixing them with tagged values
- «stereotype» operation(...){ tag1 = value1, tag2 = value2, ...}

Query operations -

- Each operation has a property called isQuery
- If you set this property to true in your modeling tool, the operation is a query operation
- The default value for isQuery is false

Example



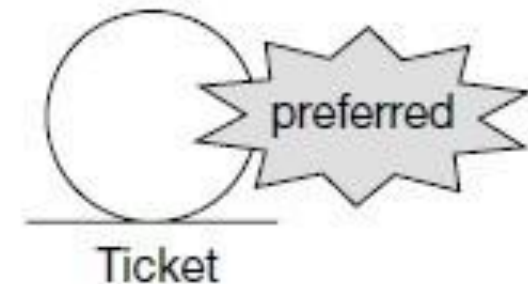
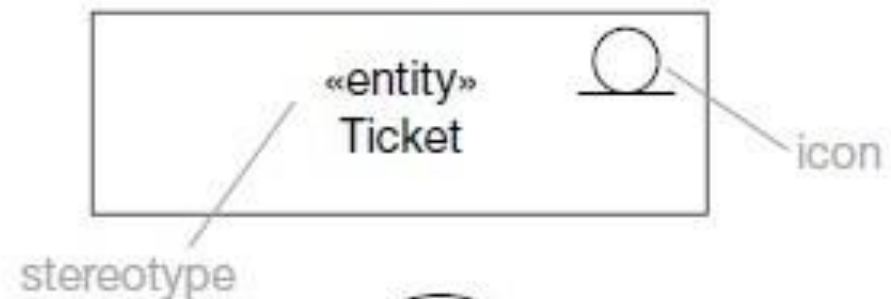
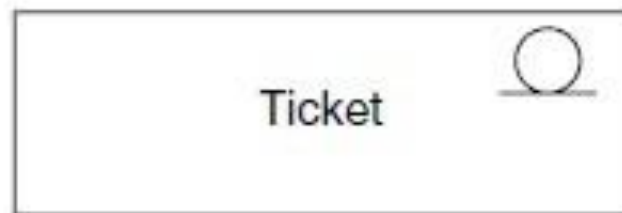
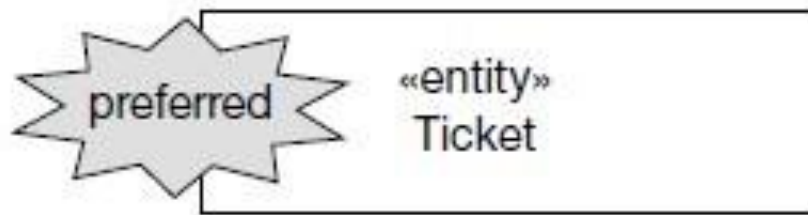
Query operation



«query» getBalance() : Double { isQuery=true }

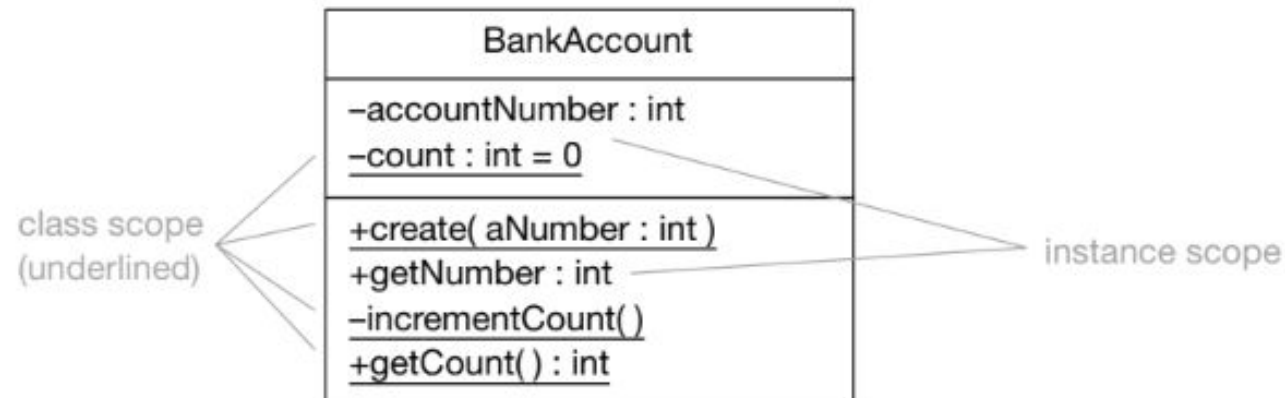
Class stereotype syntax

- There is a lot of flexibility in how stereotypes can be displayed
- However, most modellers just use the name in guillemots («stereotypeName») or the icon



Instance scope and class scope

	Instance scope	Class scope
Attributes	By default, attributes have instance scope Every object of the class gets its own copy of the instance scope attributes Each object may therefore have different instance scope attribute values	Attributes may be defined as class scope Every object of the class shares the same, single copy of the class scope attributes Each object will therefore have the same class scope attribute values
Operations	By default, operations have instance scope Every invocation of an instance scope operation applies to a specific instance of the class You can't invoke an instance scope operation unless you have an instance of the class available – clearly, this means that you <i>can't</i> use an instance scope operation of a class to create objects of that class, as you could never create the first object	Operations may be defined as class scope Invocation of a class scope operation does not apply to any specific instance of the class – instead, you can think of class scope operations as applying to the class itself You can invoke a class scope operation even if there is no instance of the class available – this is ideal for object creation operations



Object construction and destruction

Constructors : Constructors are special methods used to create and initialize objects of a class.

They have class scope, because you need to call them even before an object exists.

Destructor : Destructors are special methods used to clean up resources when an object is destroyed.

BankAccount
<u>+create(aNumber : int)</u>

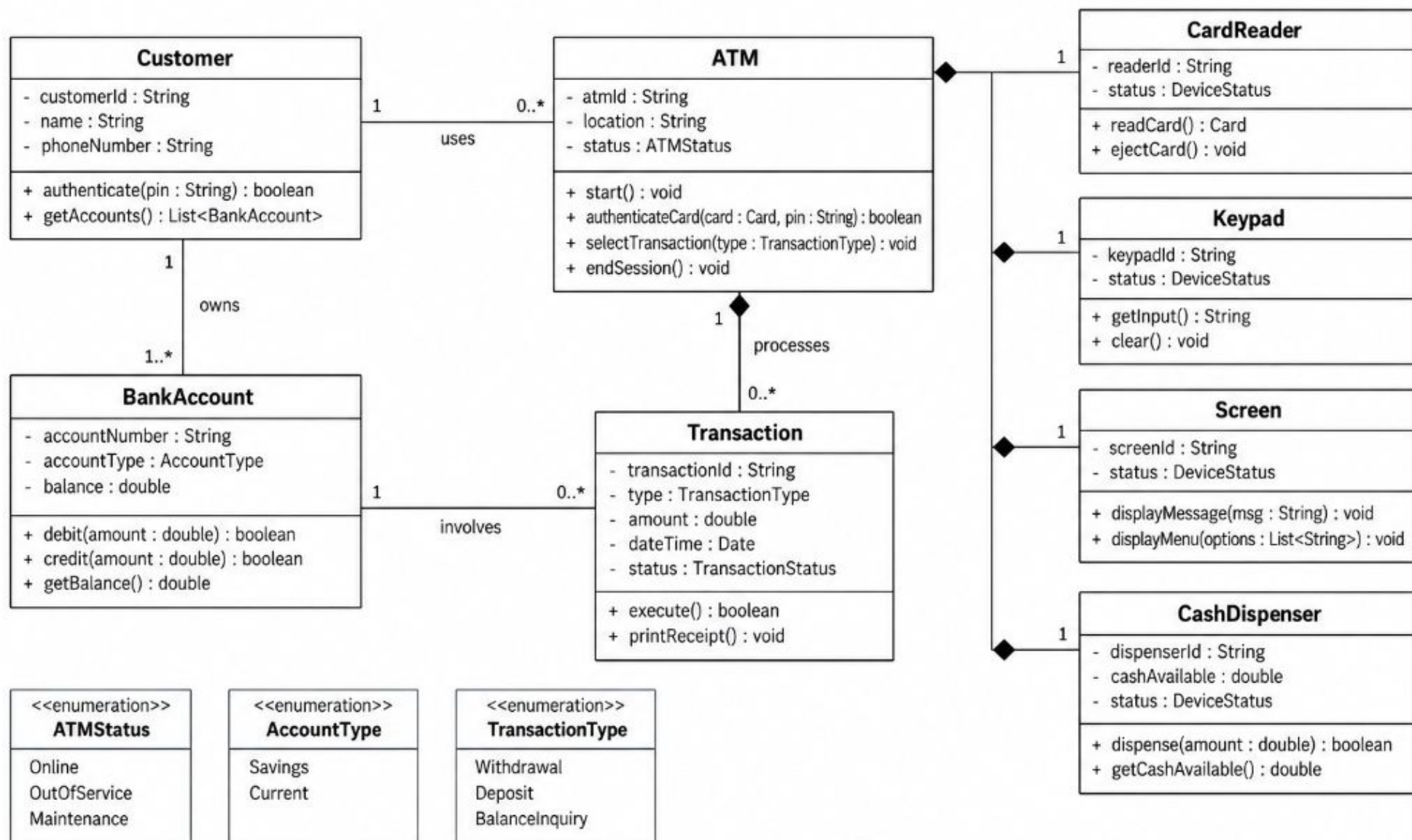
Generic constructor name

BankAccount
<u>+BankAccount(aNumber : int)</u>

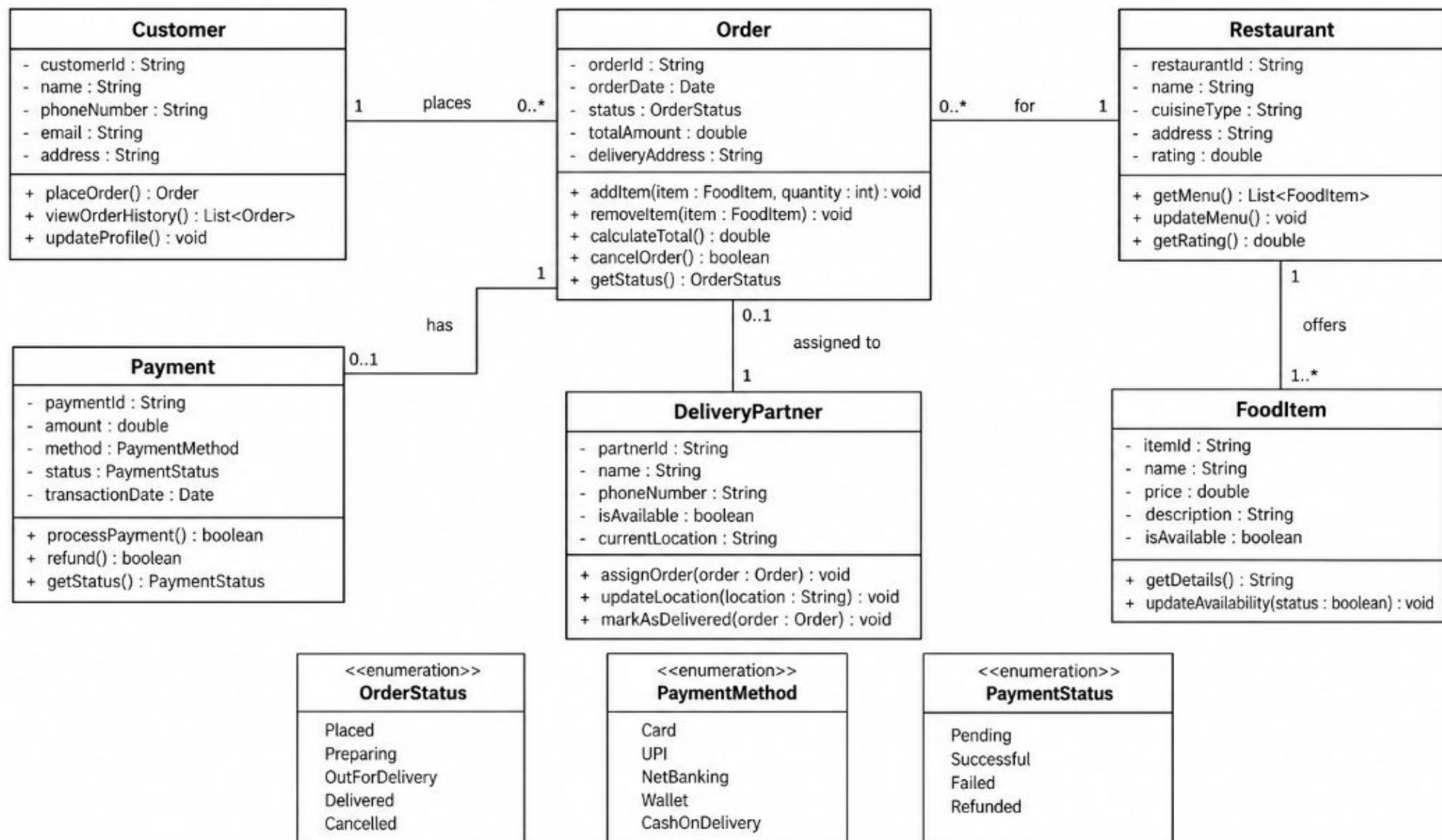
Java/C#/C++ standard

BankAccount
-accountNumber : int <u>-count : int = 0</u>
<u>+create(aNumber : int)</u> +getNumber() : int <u>-incrementCount()</u> <u>+getCount() : int</u>

Example - ATM



Example - Online Food Delivery



THANK YOU